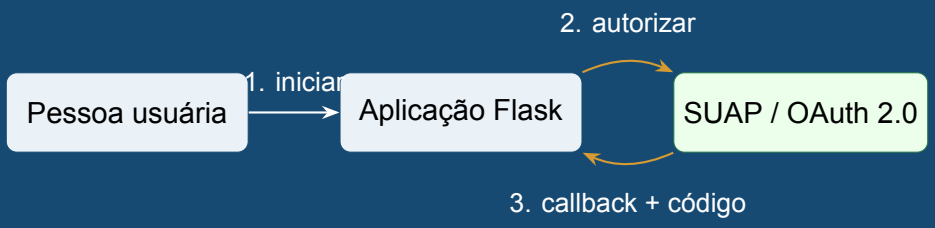
DESENVOLVIMENTO WEB

Tutorial de integração com o SUAP

Login OAuth 2.0 e consumo da API em uma aplicação Flask

Guia prático do cadastro ao primeiro login



Exemplo didático com Flask e OAuth 2.0

Novembro de 2026

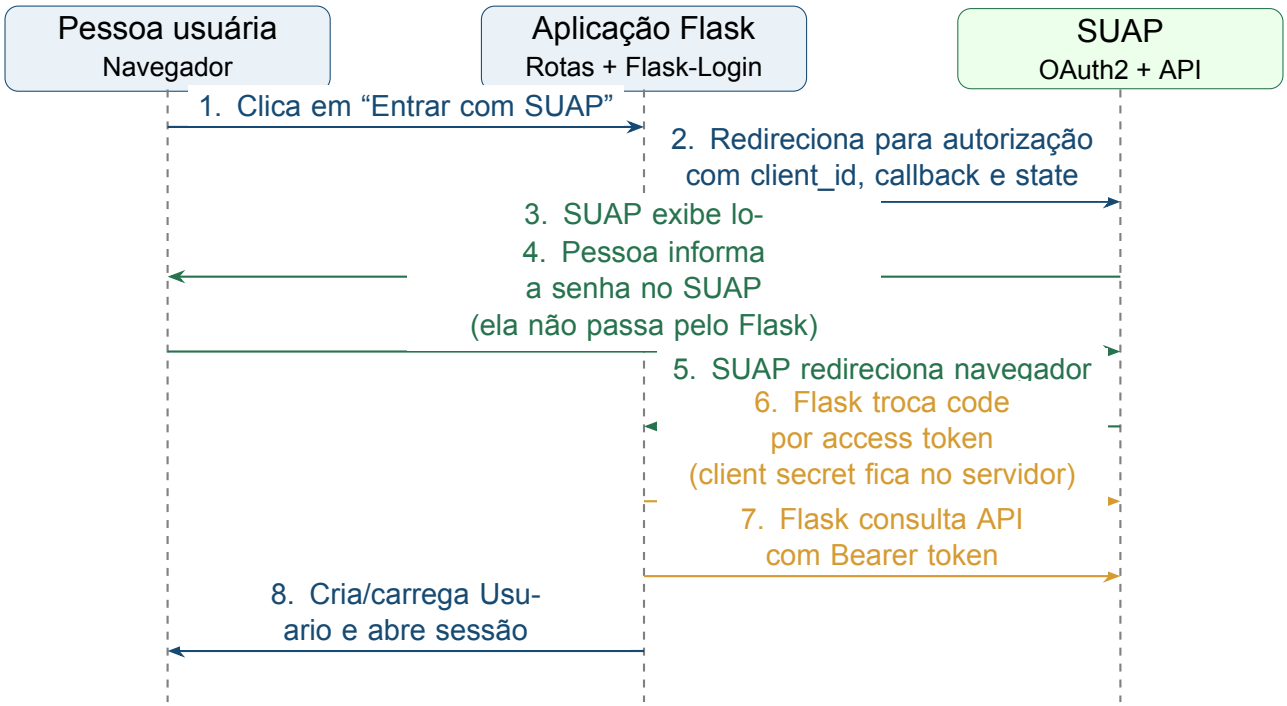
1. Como organizar a integração

Visão do exemplo. O tutorial mostra uma implementação de referência em Flask: rotas, serviço OAuth, consulta ao perfil e vínculo local via `suap_id`. Adapte os nomes de módulos e configurações ao seu aplicativo.

O fluxo fica dividido por responsabilidade: as rotas iniciam OAuth e recebem o callback; um serviço monta URLs, troca o código por token e consulta o perfil; outro serviço associa o perfil a um usuário local e chama Flask-Login; o modelo guarda o vínculo; e o template apresenta o botão de entrada. A migration acrescenta a coluna sem apagar os usuários existentes.

Visão geral do fluxo de autenticação

O diagrama abaixo mostra o percurso completo da autorização até a sessão Flask-Login. Leia de cima para baixo: o navegador transporta a pessoa entre os sites, enquanto a troca do código por token acontece diretamente entre os servidores.



Leitura rápida: a senha do SUAP é digitada somente no site do SUAP. O Flask recebe um código temporário; o access token e o client secret são tratados no back-end.

Figura 1 — Fluxo OAuth2 Authorization Code integrado à sessão local do Flask.

2. Preparação: conta, URLs e tipo de cliente

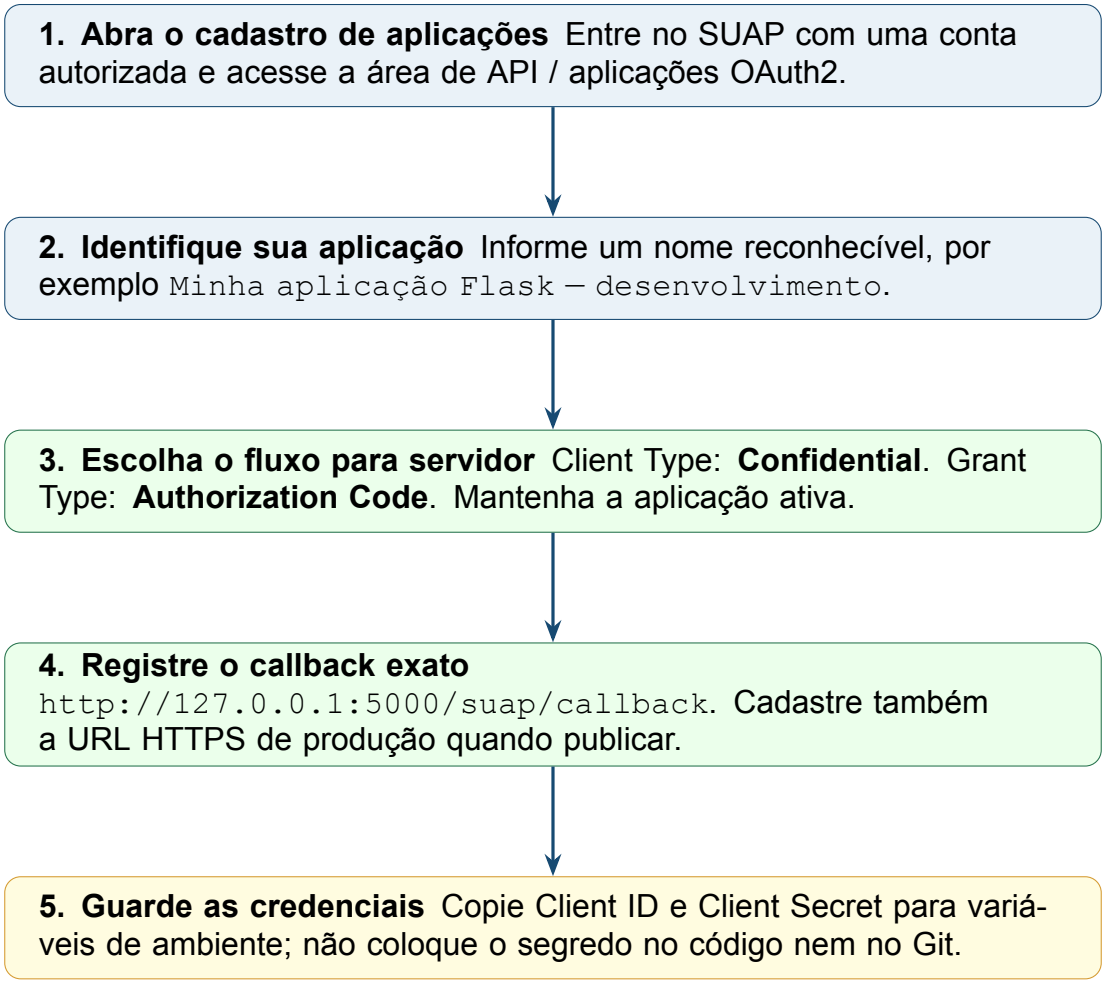
É necessário ter uma conta ativa no SUAP/IFRN com permissão para registrar aplicações. Acesse `https://suap.ifrn.edu.br/api/` e crie uma aplicação OAuth2 (o painel pode pedir autenticação). Use estes valores para uma aplicação Flask executada localmente:

- Nome: por exemplo, `Minha aplicação Flask (desenvolvimento)`.
- Tipo de cliente: **Confidential**, pois o segredo ficará no servidor Flask.
- Tipo de concessão (grant): **Authorization Code**.
- URI de redirecionamento: `http://127.0.0.1:5000/suap/callback`.
- Ativo: habilitado; suporte OIDC/algoritmo, se exibido, deve seguir a configuração suportada pelo SUAP e não é necessário para OAuth2 básico.

Cadastre a URI exatamente como será usada no Flask: host, porta, caminho e barra final precisam coincidir. Se testar com `localhost`, cadastre essa URI também; não presuma equivalência entre `localhost` e `127.0.0.1`. Para hospedagem, acrescente a URL HTTPS pública do callback. Anote `Client ID` e `Client Secret` no momento da criação; trate o segredo como senha e não o coloque no Git.

Cadastro no painel SUAP: sequência visual

Use esta sequência junto à tela de cadastro em `https://suap.ifrn.edu.br/api/`. Os nomes dos campos podem variar um pouco; os valores do exemplo devem corresponder ao ambiente em que o Flask está rodando.



Atenção à URI: host, protocolo, porta, caminho e barra final precisam coincidir literalmente. `localhost` e `127.0.0.1` são endereços diferentes para a validação do callback.

Figura 2 — Cadastro de um cliente OAuth2 confidencial para desenvolvimento local.

3. Configurar credenciais e preparar o banco

Esta etapa parte de um clone limpo. Instale Python 3.12 ou superior e MySQL Server. Se você já recebeu a pasta do projeto, comece na raiz `app_flask`. No comando abaixo, substitua o endereço de exemplo pelo endereço Git fornecido pela turma:

```
git clone URL_DO_REPOSITORIO app_flask
cd app_flask
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

No Windows, crie o ambiente virtual:

```
py -m venv .venv
```

No PowerShell, ative-o e instale as dependências:

```
.\.venv\Scripts\Activate.ps1
py -m pip install --upgrade pip
pip install -r requirements.txt
```

No Prompt de Comando (cmd.exe), use:

```
.venv\Scripts\activate.bat
py -m pip install --upgrade pip
pip install -r requirements.txt
```

O serviço SUAP usa `urllib`, biblioteca padrão do Python. Não é necessário instalar `requests` nem uma dependência OAuth adicional.

O pacote `mysql-connector-python` instalado pelo pip é o driver de conexão; ele não instala o servidor MySQL. Instale e inicie o MySQL Server no sistema operacional antes de criar o banco. Escolha um nome de banco e use o mesmo valor na configuração `DB_NAME` do aplicativo. Neste exemplo, o banco se chama `suap_flask_app`:

```
mysql -u root -p
```

No prompt do MySQL, execute:

```
CREATE DATABASE `suap_flask_app` CHARACTER SET utf8mb4;
EXIT;
```

O projeto carrega `.env` com `load_dotenv()` em `config.py`; o `.gitignore` já exclui esse arquivo. Crie e edite-o na raiz:

```
nano .env
```

No Windows, abra o arquivo com `notepad .env`. Gere uma chave secreta de sessão:

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Copie o resultado e as credenciais da aplicação registrada no SUAP para o arquivo `.env`. Ajuste usuário e senha de MySQL se necessário:

```
SECRET_KEY=cole_a_chave_gerada
DB_USERNAME=root
DB_PASSWORD=sua_senha_mysql
SUAP_URL=https://suap.ifrn.edu.br
SUAP_CLIENT_ID=cole_aqui_o_client_id
SUAP_CLIENT_SECRET=cole_aqui_o_client_secret
```

As configurações correspondentes em `config.py` são:

```
SECRET_KEY = os.getenv('SECRET_KEY', 'chave-secreta')
DB_PASSWORD = os.getenv("DB_PASSWORD", "labinfo")
DB_USERNAME = os.getenv("DB_USERNAME", "root")
SUAP_URL = os.getenv('SUAP_URL', 'https://suap.ifrn.edu.br')
SUAP_CLIENT_ID = os.getenv('SUAP_CLIENT_ID')
SUAP_CLIENT_SECRET = os.getenv('SUAP_CLIENT_SECRET')
```

No arquivo `app/models/usuario.py`, dentro da classe `Usuario`, adicione o campo abaixo. Ele é opcional para preservar as contas locais anteriores:

```
suap_id = db.Column(db.String(64), unique=True, nullable=True)
```

Com MySQL ativo, o banco criado e as credenciais em `.env`, crie a migration para registrar a alteração do modelo e aplique-a:

```
flask db migrate -m "adiciona suap_id ao usuario"
flask db upgrade
flask db current
```

Abra o arquivo criado em `migrations/versions/` e confirme que ele adiciona `suap_id` como anulável e com restrição única. Neste branch atualizado, a migration já existe: execute somente `flask db upgrade`, sem gerar outra migration. A pasta `migrations/` já existe, portanto não execute `flask db init`. Em um banco vazio, `upgrade` também cria as tabelas iniciais de usuário e postagem. `suap_id` fica nulo nas contas locais já existentes. Ao entrar pelo SUAP, o serviço associa a conta pelo identificador informado no perfil. A chave gerada e salva como `SECRET_KEY` no `.env` substitui o valor de desenvolvimento padrão de `config.py`; não exponha o valor padrão em produção.

4. Como o serviço conversa com o SUAP

O serviço do projeto está em `app/services/SuapAuthenticationService.py`. Ele monta a URL de autorização, troca o código OAuth por token usando `client_id` e `client_secret`, e consulta o perfil com Bearer token. Abaixo está o código atual: Se estiver começando a partir de uma versão do projeto sem esse serviço, crie o arquivo no local indicado antes de copiar o código:

```
touch app/services/SuapAuthenticationService.py
```

```
import json
from urllib.error import URLError
from urllib.parse import urlencode
from urllib.request import Request, urlopen

from app import app


class SuapAuthenticationService:
   @staticmethod
   def authorization_url(state, redirect_uri):
      params = urlencode({
         'response_type': 'code',
         'client_id': app.config['SUAP_CLIENT_ID'],
         'redirect_uri': redirect_uri,
         'state': state,
      })
      return f"{app.config['SUAP_URL']}/o/authorize/?{params}"

   @staticmethod
   def exchange_code(code, redirect_uri):
      data = urlencode({
         'grant_type': 'authorization_code',
         'code': code,
         'redirect_uri': redirect_uri,
         'client_id': app.config['SUAP_CLIENT_ID'],
         'client_secret': app.config['SUAP_CLIENT_SECRET'],
      }).encode()
      try:
         with urlopen(Request(
            f"{app.config['SUAP_URL']}/o/token/",
            data=data,
            headers={'Content-Type': 'application/x-www-form-urlencoded'},
         )) as response:
            return json.load(response)
      except URLError:
         return None

   @staticmethod
   def meus_dados(access_token):
```

```
    request = Request(
        f"{app.config['SUAP_URL']}/api/rh/meus-dados/",
        headers={
            'Authorization': f'Bearer {access_token}',
            'Accept': 'application/json',
        },
    )
    try:
        with urlopen(request) as response:
            return json.load(response)
    except URLError:
        return None
```

Os caminhos usados pelo projeto são `/o/authorize/`, `/o/token/` e `/api/rh/meus-dados/`. Para outra instalação SUAP, altere `SUAP_URL` e confira os campos do perfil na documentação interativa daquele ambiente.

5. Rotas Flask: iniciar, validar e concluir o login

O link do botão em `app/templates/login.html` aponta para a rota `login_suap`:

```
<a class="btn btn-secondary btn-user btn-block" href="{{ url_for('login_suap')
    }}">
    Entrar com SUAP
</a>
```

Em `app/routes.py`, a rota de início cria um `state` aleatório, guarda-o na sessão e encaminha o navegador ao SUAP. A URI de callback é montada a partir do host e da rota atual; ela precisa ser idêntica à cadastrada no painel: As importações abaixo já existem no arquivo atual e são necessárias para as rotas apresentadas:

```
from app import app
from flask import render_template, redirect, flash, request, url_for, session
from secrets import token_urlsafe
from urllib.parse import urljoin
from app.services.AuthenticationService import AuthenticationService
from app.services.SuapAuthenticationService import SuapAuthenticationService
```

```
@app.route('/login-suap')
def login_suap():
    state = token_urlsafe(32)
    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    session['suap_oauth_state'] = state
    return redirect(SuapAuthenticationService.authorization_url(state,
        redirect_uri))
```

O callback valida o `state`, troca o código, busca os dados do perfil, vincula o usuário e exibe a página `suap_eu.html`. O token fica em memória apenas durante a chamada e não é gravado no cookie de sessão:

```
@app.route('/suap/callback')
def suap_callback():
    state = session.pop('suap_oauth_state', None)
    if request.args.get('error') or not state or state != request.args.get('state
        ', ''):
        flash('Falha na autenticação do SUAP.', 'error')
        return redirect(url_for('login'))

    code = request.args.get('code')
    if not code:
```

```
        flash('Código de autorização do SUAP não recebido.', 'error')
        return redirect(url_for('login'))

    redirect_uri = request.host_url.rstrip('/') + url_for('suap_callback')
    tokens = SuapAuthenticationService.exchange_code(code, redirect_uri)
    if not tokens or not tokens.get('access_token'):
        flash('Não foi possível obter o token do SUAP.', 'error')
        return redirect(url_for('login'))

    dados = SuapAuthenticationService.meus_dados(tokens['access_token'])
    if dados is None:
        flash('Não foi possível consultar os dados do SUAP.', 'error')
        return redirect(url_for('login'))

    usuario = AuthenticationService.login_suap(dados)
    if usuario is None:
        flash('Não foi possível vincular esta conta SUAP a um usuário local.', '
            error')
        return redirect(url_for('login'))

    foto = dados.get('url_foto_75x100') or dados.get('url_foto') or dados.get('
        foto')
    foto_url = urljoin(app.config['SUAP_URL'] + '/', foto) if foto else None
    return render_template('suap_eu.html', title='Perfil SUAP', dados=dados,
        foto_url=foto_url)
```

O `state` vincula o início ao retorno OAuth e evita aceitar callbacks iniciados fora da sessão. Se o SUAP recusar o acesso, retornar sem código ou falhar ao consultar a API, a rota informa o erro e volta à tela de login. Depois do login, `suap_eu.html` mostra nome, vínculo, matrícula, e-mail, campus e foto quando esses campos estiverem disponíveis.

6. Associação do perfil e persistência em `suap_id`

O usuário local continua sendo um modelo `Usuario` do SQLAlchemy e o Flask-Login mantém a sessão. Em `app/services/AuthenticationService.py`, o valor de `suap_id` vem de matrícula, username ou e-mail do SUAP; o username local vem de nome usual ou nome. O serviço primeiro procura por `suap_id`; se ainda não houver vínculo, procura uma conta local compatível e associa-a:

```
from app import db
from app.models.usuario import Usuario
from flask_login import login_user, logout_user
from werkzeug.security import check_password_hash


class AuthenticationService():
    def login(form):
        username = form.username.data.strip()
        user = Usuario.query.filter_by(username=username).first()

        if user and user.password_hash:
            if check_password_hash(user.password_hash, form.password.data):
                login_user(user, remember=form.remember_me.data)
                return True
        return False

    def logout():
        return logout_user()

    def login_suap(dados):
```

```
    suap_id = str(
        dados.get('matricula') or dados.get('username') or dados.get('email')
            or ''
    ).strip()
    if not suap_id:
        return None

    username = str(dados.get('nome_usual') or dados.get('nome') or '').strip()
    email = dados.get('email') or f'{username}@suap.local'
    user = Usuario.query.filter_by(suap_id=suap_id).first()

    if not user:
        user = Usuario.query.filter(
            (Usuario.username == username) | (Usuario.email == email)
        ).first()

    if user and user.suap_id and user.suap_id != suap_id:
        return None

    if not user:
        user = Usuario(username=username, email=email, suap_id=suap_id)
        db.session.add(user)
    else:
        user.suap_id = suap_id

    try:
        db.session.commit()
    except Exception:
        db.session.rollback()
        return None

    login_user(user)
    return user
```

O modelo declara o campo assim:

```
suap_id = db.Column(db.String(64), unique=True, nullable=True)
```

O campo aceita `NULL` para não exigir uma identidade SUAP das contas criadas pela aplicação. A restrição `unique=True` evita associar o mesmo identificador remoto a mais de uma conta. A senha SUAP não é copiada para `password_hash`; a autenticação remota é validada pelo protocolo OAuth e a sessão local é aberta por `login_user(user)`.

A migration deve ser gerada por cada aluno a partir do modelo atualizado. Execute os comandos a partir da raiz do projeto:

```
flask db migrate -m "adiciona suap_id ao usuario"
flask db upgrade
```

O Alembic cria automaticamente um identificador de revisão para o arquivo. Esse identificador pode variar entre cópias; não o escreva manualmente nem copie o de outra pessoa. Antes de aplicar a migration, confira se ela adiciona uma coluna anulável e única. Neste checkout, a migration já está incluída, então basta executar `flask db upgrade`.

7. Executar e percorrer o fluxo completo

Ative o ambiente virtual e instale as dependências declaradas no aplicativo. Confirme que o MySQL está ativo, que o banco escolhido existe e que o arquivo `.env` contém as credenciais da aplicação. Aplique a migration adicionada e inicie o Flask na mesma porta e host usados no cadastro SUAP:

```
flask db upgrade
flask run --host 127.0.0.1 --port 5000 --debug
```

Abra `http://127.0.0.1:5000/login`, clique em “Entrar com SUAP”, autentique-se no domínio do SUAP e autorize o acesso. O navegador retorna a `/suap/callback`; a aplicação valida `state`, troca o código por token, chama `/api/rh/meus-dados/`, cria ou encontra o registro com `suap_id`, inicia sessão Flask-Login e mostra `suap_eu.html`. Confirme o nome e os dados do perfil; em seguida, teste sair por `/logout` e entrar novamente pelo botão SUAP.

8. Diagnóstico rápido

- **redirect URI inválida**: o callback usa host da requisição e rota `suap_callback`. Compare protocolo, host, porta e caminho com o cadastro no SUAP.
- **client inválido**: confira se Client ID/Secret correspondem à aplicação ativa e se o segredo não contém espaços adicionais.
- **state inválido**: início e callback precisam compartilhar a mesma sessão/cookie e chave secreta Flask; não inicie o callback diretamente sem começar pelo botão.
- **token aceito, perfil negado**: confira o endpoint `/api/rh/meus-dados/`, as permissões da conta e a documentação interativa do SUAP. O serviço envia o token como `Authorization: Bearer ...`.
- **funciona em localhost, falha publicado**: cadastre a URL HTTPS pública do callback e configure proxy/HTTPS corretamente; a rota deriva o callback de `request.host_url`.
- **usuário autentica, mas áreas protegidas redirecionam**: confira vínculo ao modelo `Usuario`, chamada a `login_user` e função `load_user`.

9. Referências e observação de segurança

- Documentação interativa oficial: `https://suap.ifrn.edu.br/api/docs/`.
- Cadastro e acesso à API: `https://suap.ifrn.edu.br/api/`.
- Cliente oficial IFRN para acesso à API: `https://ifrn.github.io/suapi/`.
- Referência IFRN de aplicação OAuth2 Confidential + Authorization Code: `https://github.com/ifrn-oficial/cliente_suap_django`.

Os formulários e rótulos do painel podem mudar. A documentação interativa do SUAP é a fonte final para endpoints, parâmetros e permissões vigentes. O código consulta o endpoint de perfil configurado no serviço e descarta o access token ao final da requisição; nunca compartilhe Client Secret, access token ou cookies de sessão.